Image Upload & Automatic Thumbnail Generation

A Serverless, Event-Driven Architecture Using PHP, Amazon S3, and AWS Lambda

College Project Proposal

August 24, 2026

Contents

1 Overview

1.1 Introduction

The proposed project, **Image Upload & Automatic Thumbnail Generation**, is designed to demonstrate how a traditional PHP-based web application can be integrated with modern cloud infrastructure to automate a common but resource-intensive task: generating thumbnail versions of user-uploaded images. Rather than performing image resizing directly on the web server, the proposed system offloads this work to a serverless cloud pipeline built on Amazon Web Services (AWS).

1.2 Problem Addressed

Many web applications allow users to upload images, but generating thumbnails for those images is often handled synchronously on the same server that serves the website. This approach has several drawbacks that the proposed system intends to address:

- Image processing consumes server CPU and memory, which can slow down the main web application during periods of heavy upload activity.
- Thumbnail generation logic is tightly coupled with the web server, making the system harder to scale independently.
- Manually managing multiple image sizes and formats on a single server increases maintenance complexity.

Automatic thumbnail generation is useful because it improves page load times, reduces bandwidth usage, and provides a consistent, smaller preview of an image without requiring the browser to download the full-resolution file. By generating thumbnails automatically and asynchronously, the website can remain responsive while the actual image processing happens elsewhere.

1.3 Proposed Solution

The proposed system separates the concerns of *uploading* an image and *processing* that image. The PHP website is responsible only for accepting, validating, and forwarding the uploaded image to cloud storage. Once the image is stored, a cloud-native, event-driven pipeline automatically detects the new upload and generates a thumbnail, without any additional involvement from the PHP server.

1.4 Main Technologies

The proposed architecture makes use of the following core technologies:

- **PHP** – for handling the website backend and image upload logic.
- **Amazon S3** – for durable, scalable object storage of both original images and generated thumbnails.
- **AWS Lambda** – for serverless execution of the thumbnail generation logic.
- **Python with the Pillow library** – for the actual image processing (resizing) inside the Lambda function.

1.5 Overall Architecture

At a high level, the proposed workflow is as follows: a user selects an image in the browser, which is submitted to the PHP website. PHP validates the image and uploads it to an Amazon S3 bucket using the AWS SDK for PHP. When S3 detects that a new object has been created, it automatically emits an *Object Created* event. This event triggers an AWS Lambda function, which uses Python and the Pillow library to open the uploaded image, resize it into a thumbnail, and upload the resulting thumbnail back into the same S3 bucket. The browser can then retrieve and display the generated thumbnail.

This design is intentionally *event-driven*: no component needs to poll or actively coordinate with another beyond reacting to the events it is configured to observe. Figure 1 illustrates this proposed flow.

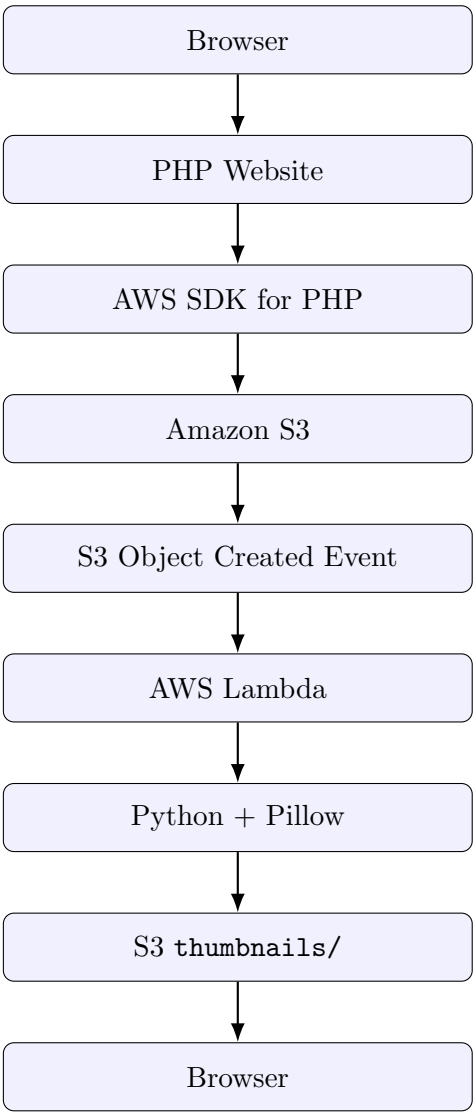

Figure 1: Proposed high-level architecture of the image upload and thumbnail generation pipeline.

This section has introduced the project at a conceptual level. The next section examines the design of each component in greater technical depth.

2 More Nuanced Perspective

This section presents a deeper technical discussion of the proposed system, examining the responsibilities of each component, the reasoning behind key design decisions, and the security considerations that inform the architecture.

2.1 PHP Website

In the proposed design, PHP acts purely as an upload gateway and does not perform any image processing itself. Its responsibilities are limited to:

- Receiving the uploaded image from the browser via an HTTP form submission.
- Validating that the uploaded file is genuinely an image, rather than trusting the file extension alone.
- Checking the MIME type of the uploaded file against an allow-list of accepted image formats (for example, JPEG and PNG).
- Checking the file size against a configured maximum limit to prevent excessively large uploads.
- Generating a unique filename for the image (for example, using a UUID or hash) so that uploads from different users cannot collide or overwrite one another.
- Uploading the validated image to the appropriate location in the Amazon S3 bucket using the AWS SDK for PHP.

By keeping PHP's responsibilities narrow, the proposed system avoids duplicating image-processing logic on the web server and keeps the upload path lightweight.

2.2 Amazon S3

Amazon S3 is proposed as the central storage layer for both original images and their generated thumbnails. Rather than using two separate buckets, the design proposes a single bucket organized using two prefixes, which behave similarly to folders:

- `uploads/` – stores the original, unprocessed images submitted by users.
- `thumbnails/` – stores the resized thumbnail versions generated by the Lambda function.

An example of the proposed bucket layout is shown below:

```
my-bucket/
+-- uploads/
|   +-- image1.jpg
|   +-- image2.png
|
+-- thumbnails/
    +-- image1_thumb.jpg
    +-- image2_thumb.jpg
```

A key design decision is that the S3 event notification is configured to trigger *only* for objects created under the `uploads/` prefix. This is intentional: if the event were configured

for the entire bucket, then the act of Lambda writing a thumbnail into `thumbnails/` would itself generate a new "Object Created" event, which would attempt to trigger Lambda again. Restricting the event trigger to the `uploads/` prefix prevents this self-triggering (infinite loop) scenario and ensures Lambda only executes in response to genuine new uploads.

2.3 AWS Lambda

AWS Lambda is proposed as the compute layer responsible for all image processing. Because Lambda is serverless, the proposed system does not require provisioning or managing any dedicated processing server; the function runs only when triggered by an S3 event and scales automatically with upload volume.

The proposed Lambda function is designed to perform the following steps when invoked:

1. Receive the S3 event payload describing the newly created object.
2. Identify the uploaded image's bucket name and object key from the event data.
3. Download the image from the `uploads/` prefix into the Lambda execution environment.
4. Open the image using the Pillow library.
5. Resize the image to fit within a maximum thumbnail dimension.
6. Maintain the original aspect ratio while resizing, to avoid visual distortion.
7. Generate the final thumbnail image.
8. Upload the resulting thumbnail back to S3 under the `thumbnails/` prefix.

The proposed maximum thumbnail size is approximately **300 × 300 pixels**. Because the resize operation is designed to preserve aspect ratio, an image is scaled to fit within this bounding box rather than being stretched to fill it exactly. For example, an original image of size 1200×800 pixels would be expected to produce a thumbnail of approximately 300×200 pixels, rather than a distorted 300×300 image.

2.3.1 Lambda Function Implementation

To make the design above concrete, this subsection presents a proposed Python implementation of the AWS Lambda function, followed by a walkthrough of how each part of the code corresponds to the processing steps outlined earlier. The function is written for the standard AWS Lambda Python runtime and depends on `boto3` (included in the Lambda runtime by default) and `Pillow` (supplied via a Lambda Layer or included in the deployment package).

Listing 1: Proposed `lambda_function.py` for thumbnail generation.

```
"""
lambda_function.py

AWS Lambda function that generates thumbnails from uploaded images.

Trigger: S3 ObjectCreated event on prefix: uploads/
Input: S3 object under uploads/
Output: JPEG thumbnail under thumbnails/ (max 300x300, aspect ratio preserved)
```

```
9
10 Dependencies: boto3 (built into Lambda), Pillow (via Lambda Layer or deployment ZIP)
11 """
12
13 import os
14 import io
15 import logging
16 import urllib.parse
17
18 import boto3
19 from PIL import Image
20
21 # Logging
22 logger = logging.getLogger()
23 logger.setLevel(logging.INFO)
24
25 # Constants
26 UPLOADS_PREFIX = 'uploads/'
27 THUMBNAILS_PREFIX = 'thumbnails/'
28 THUMB_SIZE = (300, 300) # max width, max height - aspect ratio preserved
29 THUMB_SUFFIX = '_thumb'
30 THUMB_FORMAT = 'JPEG'
31 THUMB_QUALITY = 85 # JPEG quality 1-95
32
33 s3 = boto3.client('s3')
34
35
36 def lambda_handler(event, context):
37     """
38     Entry point for S3-triggered Lambda.
39     Iterates over all S3 records in the event (usually 1).
40     """
41     logger.info("Received event: %s", event)
42
43     for record in event.get('Records', []):
44         try:
45             process_record(record)
46         except Exception as exc:
47             # Log but do not re-raise so one bad record does not block others.
48             logger.error("Error processing record %s: %s", record, exc, exc_info=True)
49
50     return {'statusCode': 200, 'body': 'OK'}
51
52
53 def process_record(record):
54     """Process a single S3 event record."""
55     bucket = record['s3']['bucket']['name']
56     raw_key = record['s3']['object']['key']
57     key = urllib.parse.unquote_plus(raw_key)
58
59     logger.info("Processing: s3://%s/%s", bucket, key)
60
61     # Safety check: only process uploads/ prefix
62     if not key.startswith(UPLOADS_PREFIX):
63         logger.warning("Skipping key outside uploads/ prefix: %s", key)
64         return
65
66     # Derive thumbnail key
```

```
    filename_with_ext = key[len(UPLOADS_PREFIX):]
    base, _ext = os.path.splitext(filename_with_ext)
    thumb_filename = base + THUMB_SUFFIX + '.jpg'
    thumb_key = THUMBNAILS_PREFIX + thumb_filename

    logger.info("Thumbnail key will be: %s", thumb_key)

    # Download original image from S3
    response = s3.get_object(Bucket=bucket, Key=key)
    image_data = response['Body'].read()
    logger.info("Downloaded %d bytes from S3", len(image_data))

    # Open with Pillow
    image = Image.open(io.BytesIO(image_data))
    original_size = image.size
    logger.info("Original image size: %dx%d, mode: %s", *original_size, image.mode)

    # Handle animated GIF: use first frame only
    if getattr(image, 'is_animated', False):
        image.seek(0)

    # Convert transparent/palette modes to RGB for JPEG output
    if image.mode in ('RGBA', 'LA'):
        background = Image.new('RGB', image.size, (255, 255, 255))
        if image.mode == 'RGBA':
            background.paste(image, mask=image.split()[3])
        else:
            background.paste(image.convert('RGBA'), mask=image.convert('RGBA').split()
                [3])
        image = background
    elif image.mode == 'P':
        image = image.convert('RGBA')
        background = Image.new('RGB', image.size, (255, 255, 255))
        background.paste(image, mask=image.split()[3])
        image = background
    elif image.mode != 'RGB':
        image = image.convert('RGB')

    # Generate thumbnail (aspect-ratio preserved, no stretching)
    image.thumbnail(THUMB_SIZE, Image.LANCZOS)
    thumb_size = image.size
    logger.info(
        "Thumbnail size: %dx%d (original was %dx%d)",
        thumb_size[0], thumb_size[1],
        original_size[0], original_size[1],
    )

    # Save thumbnail to in-memory buffer
    buffer = io.BytesIO()
    image.save(buffer, format=THUMB_FORMAT, quality=THUMB_QUALITY, optimize=True)
    buffer.seek(0)

    # Upload thumbnail to S3
    s3.put_object(
        Bucket = bucket,
        Key = thumb_key,
        Body = buffer,
        ContentType = 'image/jpeg',
```

```
        Metadata = {
            'original-key': key,
            'original-width': str(original_size[0]),
            'original-height': str(original_size[1]),
            'thumb-width': str(thumb_size[0]),
            'thumb-height': str(thumb_size[1]),
        },
    )

    logger.info(
        "Thumbnail uploaded successfully to s3://%s/%s (%dx%d)",
        bucket, thumb_key, thumb_size[0], thumb_size[1],
    )
```

2.3.2 Code Walkthrough

The listing above is organized into two functions, `lambda_handler` and `process_record`, plus a small block of module-level configuration. Each part is explained below.

Configuration and Constants. At the top of the file, a set of constants defines the behavior of the function: `UPLOADS_PREFIX` and `THUMBNAILS_PREFIX` identify the two S3 prefixes discussed in Section 2, `THUMB_SIZE` fixes the maximum thumbnail bounding box at 300×300 pixels, and `THUMB_QUALITY` controls the JPEG compression level used when the thumbnail is saved. A single shared `boto3` S3 client is created once at module scope so that it can be reused across multiple invocations of a warm Lambda execution environment, which is a standard performance practice for Lambda functions.

`lambda_handler` – **Entry Point.** This is the function AWS Lambda invokes directly. Because an S3 event notification can, in principle, contain more than one record, the handler loops over `event['Records']` and processes each one individually via `process_record`. Each record is wrapped in a `try/except` block so that an error while processing one uploaded file (for example, a corrupted image) is logged but does not prevent the remaining records in the same invocation from being processed. The function returns a simple status dictionary, which is sufficient for an S3-triggered invocation that does not need to return a value to a caller.

`process_record` – **Locating the Uploaded Object.** The function begins by reading the bucket name and object key out of the S3 event record. Because S3 keys can contain characters that get URL-encoded in event notifications (such as spaces becoming `+`), the key is decoded using `urllib.parse.unquote_plus` before use. As a defensive safety check, the function verifies that the key actually starts with `uploads/` and returns early otherwise – this mirrors the S3 event-prefix restriction discussed in Section 2 and provides a second layer of protection against the Lambda function ever attempting to process an object from `thumbnails/`.

Deriving the Thumbnail Key. The thumbnail's destination key is constructed by stripping the `uploads/` prefix from the original key, separating the base filename from its extension with `os.path.splitext`, and appending the `_thumb` suffix with a normalized `.jpg` extension. For an uploaded object `uploads/abc123.jpg`, this produces the destination key `thumbnails/abc123_thumb.jpg`, consistent with the example in Section 3.

Downloading and Opening the Image. The original image bytes are retrieved with `s3.get_object` and read fully into memory, then handed to `Image.open` via an in-memory `io.BytesIO` buffer – this avoids writing temporary files to the Lambda execution environment's disk. The function logs the original dimensions and color mode, which is useful both for debugging and for the CloudWatch logging described in Section 2.

Normalizing the Image Mode. Because Pillow's JPEG encoder does not support transparency, the code inspects the image's color mode and converts it to plain RGB before saving. Images with an alpha channel (`RGBA` or `LA` mode) are composited onto a solid white background so that transparent regions do not turn black or produce an error; palette-based images (`P` mode, common in PNG and GIF files) are first expanded to `RGBA` and then composited the same way; any other non-RGB mode is simply converted directly. An animated GIF is handled by seeking to its first frame with `image.seek(0)` before processing, since a thumbnail is expected to be a single static image.

Resizing While Preserving Aspect Ratio. The resize step itself uses Pillow's built-in `Image.thumbnail()` method rather than a fixed resize call. This method shrinks the image in place so that it fits within the given bounding box (`THUMB_SIZE`) while preserving the original aspect ratio, and it never enlarges an image that is already smaller than the target size. The `Image.LANCZOS` resampling filter is used to produce a high-quality, smoothly downscaled result. This is the mechanism that produces the 300×200 result described for a 1200×800 input in Section 3, rather than a distorted 300×300 square.

Saving and Uploading the Thumbnail. The resized image is written into an in-memory buffer as a JPEG at the configured quality level, again avoiding any temporary files on disk. The buffer is then uploaded to S3 with `s3.put_object` under the derived `thumbnails/` key, with its content type explicitly set to `image/jpeg`. The upload also attaches a small set of custom S3 object metadata recording the original key and both the original and thumbnail dimensions, which could later be useful to the PHP website or for auditing without needing to re-open the image.

Logging Throughout. At each major step – receiving the event, resolving the thumbnail key, downloading the original, computing the thumbnail size, and completing the upload – the function writes an informational log entry. Because Lambda automatically forwards these log statements to Amazon CloudWatch, this gives the proposed system the CloudWatch-based logging and observability referenced in Table 1, without any additional configuration.

2.4 Asynchronous Processing

Because Lambda is invoked in response to an S3 event, thumbnail generation in the proposed system is inherently **asynchronous**. There is a short delay between the moment an image is uploaded and the moment its thumbnail becomes available, since the S3 event must propagate and the Lambda function must execute and complete its upload of the thumbnail.

As a result, the thumbnail is not expected to be available immediately after the browser finishes uploading the original image. To handle this gracefully, the proposed system suggests that the browser periodically check (or *poll*) whether the thumbnail object now

exists in the `thumbnails/` prefix, and update the displayed content once it becomes available. This polling approach avoids requiring the user to manually refresh the page while still keeping the overall design simple, without introducing additional infrastructure such as WebSockets.

2.5 Security

Because the proposed system uploads user-supplied content to cloud storage and executes automated processing on it, several security considerations are incorporated into the design:

- **Server-side file validation** – all validation of uploaded files is expected to occur on the PHP server, not solely in client-side JavaScript, since client-side checks can be bypassed.
- **MIME type validation** – the actual content type of the uploaded file is checked, rather than relying on the filename extension alone.
- **File size limits** – uploads exceeding a configured maximum size are rejected before reaching S3.
- **Unique filenames** – generated filenames prevent naming collisions and reduce the risk of a malicious upload overwriting an existing file.
- **Keeping AWS credentials out of frontend code** – all AWS SDK calls are proposed to occur on the PHP server side; no AWS access keys are proposed to be exposed to the browser.
- **IAM least privilege** – the Lambda function's execution role is designed to be granted only the specific S3 permissions it requires (read from `uploads/`, write to `thumbnails/`), rather than broad account-level access.
- **Private S3 bucket** – the bucket is proposed to be kept private by default, rather than publicly readable.
- **Restricted S3 event prefix** – as discussed above, the event trigger is scoped to `uploads/` only.
- **Temporary or presigned URLs** – when displaying private images to the browser, the system proposes generating temporary, presigned S3 URLs rather than exposing the bucket contents publicly.

2.6 Event-Driven Architecture

The proposed system is considered *event-driven* because each stage of processing is triggered automatically by the completion of the previous stage, rather than being explicitly orchestrated by a central controller. The flow can be summarized as:

Image uploaded $\rightarrow$ *S3 event* $\rightarrow$ *Lambda executes automatically* $\rightarrow$ *Thumbnail generated*

This pattern offers a clear advantage: it separates the responsibility of *accepting* an upload (handled by PHP) from the responsibility of *processing* that upload (handled by Lambda). These two responsibilities can then evolve, scale, and fail independently of one another, which is expected to make the overall system easier to reason about and maintain.

3 Example Details

To make the proposed workflow concrete, this section walks through a single, complete example of what is expected to happen when a user uploads an image.

3.1 Example Scenario

Suppose a user selects a file named `vacation.jpg` to upload through the website. The proposed sequence of events is as follows:

1. The browser submits `vacation.jpg` to the PHP website.
2. PHP validates the image, checking its MIME type and file size.
3. PHP generates a unique filename for the image, for example `abc123.jpg`.
4. PHP uploads the file to S3, where it is stored as `uploads/abc123.jpg`.
5. S3 automatically generates an Object Created event for this new object.
6. The event triggers the AWS Lambda function.
7. Lambda downloads `uploads/abc123.jpg`.
8. Pillow opens and processes the image inside the Lambda function.
9. If the original image is 1200×800 pixels, the resulting thumbnail is expected to be approximately 300×200 pixels, preserving the original aspect ratio.
10. Lambda uploads the generated thumbnail to S3 as `thumbnails/abc123_thumb.jpg`.
11. The PHP website periodically checks whether the thumbnail object now exists.
12. Once the thumbnail becomes available, the browser displays it to the user.

Figure 2 illustrates this example as a step-by-step workflow diagram.

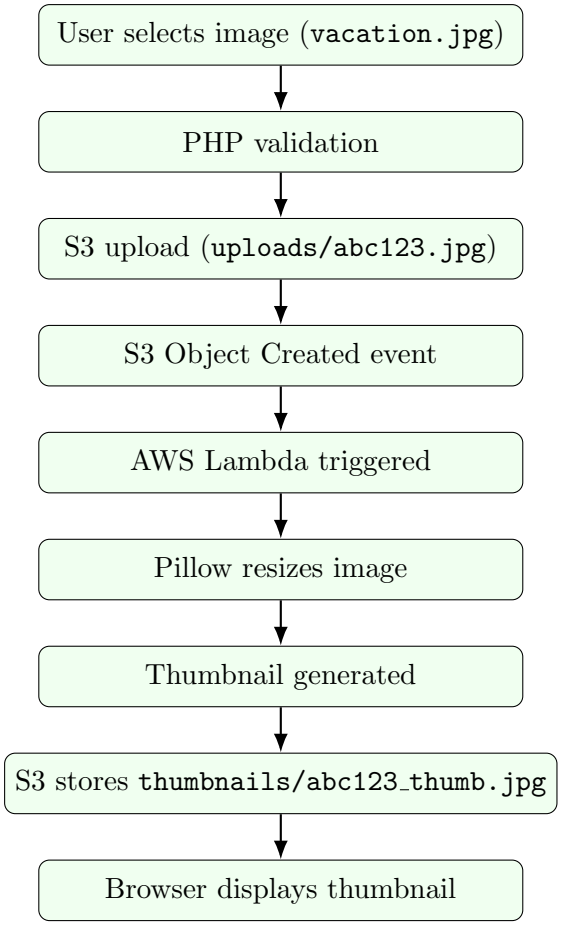

Figure 2: Example workflow triggered by a single image upload.

3.2 Technologies Used in This Example

Table 1 summarizes the technologies involved in the example above and the purpose each one serves within the proposed system.

Table 1: Technologies used in the proposed system and their purpose.

Technology	**Purpose**
PHP	Backend and upload handling
Amazon S3	Image storage
AWS Lambda	Image processing
Python	Lambda programming language
Pillow	Thumbnail generation
AWS IAM	Access control
CloudWatch	Lambda logging
JavaScript	Browser interaction

Within this example, **MIME type validation** and **unique filename generation** occur on the PHP side to ensure only safe, non-conflicting files reach S3. The **Object Created event** is the mechanism that connects the storage layer to the processing layer

without any direct communication between PHP and Lambda. Finally, **aspect-ratio-preserving resizing** in Pillow ensures the thumbnail remains visually consistent with the original image rather than appearing stretched or distorted.

4 Concluding

4.1 Summary

This proposal has outlined a system for **Image Upload & Automatic Thumbnail Generation** that demonstrates how a PHP web application can be integrated with AWS cloud services to automate image processing. Rather than performing thumbnail generation on the web server itself, the proposed architecture distributes responsibility across several purpose-built components:

- **PHP** – handles upload and validation.
- **Amazon S3** – provides storage and generates the triggering event.
- **AWS Lambda** – performs image processing.
- **Pillow** – carries out the actual thumbnail generation.
- **Amazon S3 (again)** – stores the resulting thumbnail.
- **Browser** – displays the final result to the user.

4.2 Expected Benefits

The proposed system is expected to offer the following benefits:

- **Automation** – thumbnails are generated without manual intervention.
- **Separation of responsibilities** – upload handling and image processing are decoupled.
- **Event-driven processing** – components react automatically to relevant events rather than being tightly coupled.
- **Cloud-based storage** – images are stored durably and scalably in Amazon S3.
- **Serverless image processing** – no dedicated processing server needs to be provisioned or maintained.
- **Potential scalability** – because Lambda scales automatically with the number of triggering events, the design is expected to accommodate increased upload volume without manual reconfiguration.

4.3 Future Possibilities

While the current proposal focuses on a single thumbnail size and a basic processing pipeline, several extensions could be explored in future iterations of the project. These are presented here as possibilities only, and are not part of the current proposed implementation:

- Generating multiple thumbnail sizes for different display contexts.

- Applying image compression to further reduce file size.
- Converting images to modern formats such as WebP or AVIF.
- Supporting batch processing of multiple images at once.
- Extracting image metadata (such as dimensions or EXIF data).
- Additional image optimization techniques.

These directions are not part of the current proposal but represent reasonable next steps should the project be extended beyond its initial scope.